

Creating Advanced Joins

In this chapter, you'll learn all about additional join types—what they are and how to use them. You'll also learn how to use table aliases and how to use aggregate functions with joined tables.

Using Table Aliases

In Chapter 10, “Creating Calculated Fields,” you learned how to use aliases to refer to retrieved table columns. The code to alias a column looks like this:

► Input

```
SELECT Concat(RTrim(vend_name), ' (', RTrim(vend_country), ')')
       AS vend_title
FROM vendors
ORDER BY vend_name;
```

In addition to using aliases for column names and calculated fields, SQL also enables you to alias table names. There are two primary reasons to do this:

- To shorten the SQL syntax
- To enable multiple uses of the same table within a single `SELECT` statement

Take a look at the following `SELECT` statement. It is basically the same as a statement used in the previous chapter, but it has been modified to use aliases:

► Input

```
SELECT cust_name, cust_contact
FROM customers AS c, orders AS o, orderitems AS oi
WHERE c.cust_id = o.cust_id
      AND oi.order_num = o.order_num
      AND prod_id = 'TNT2';
```

► Analysis

Notice that the three tables in the `FROM` clauses all have aliases. `customers AS c` establishes `c` as an alias for `customers`, and so on. This alias enables you to use the abbreviated `c` instead of the full text `customers`. In this example, the table aliases are used only in the

WHERE clause, but aliases are not limited to just WHERE. You can use aliases in the SELECT list, the ORDER BY clause, and any other part of a statement as well.

For the sake of comparison, here is the INNER JOIN version:

► Input

```
SELECT cust_name, cust_contact
FROM customers AS c
INNER JOIN orders AS o
    ON c.cust_id = o.cust_id
INNER JOIN orderitems AS oi
    ON oi.order_num = o.order_num
WHERE prod_id = 'TNT2';
```

► Analysis

As before, aliases are defined by using AS for each table. This time, two of them are in INNER JOIN clauses.

Note

Table Aliases Are DBMS Only It is also worth noting that table aliases are only used during query execution. Unlike column aliases, table aliases are never returned to the client.

Using Different Join Types

So far, you have used only simple joins known as inner joins, or *equijoins*. Let's now take a look at three additional join types: self-joins, natural joins, and outer joins.

Self-Joins

As mentioned earlier, one of the primary reasons to use table aliases is to be able to refer to the same table more than once in a single SELECT statement. Let's look at an example that demonstrates this.

Suppose that a problem has been found with a product (item id DTNTR), and you want a list of all the products made by that vendor so you can determine whether the same problem applies to them, too. This query requires that you first find out which vendor creates item DTNTR and then find which other products are made by the same vendor. The following is one way to approach this problem:

► Input

```
SELECT prod_id, prod_name
FROM products
WHERE vend_id = (SELECT vend_id
                  FROM products
                  WHERE prod_id = 'DTNTR');
```

► Output

prod_id	prod_name
DTNTR	Detonator
FB	Bird seed
FC	Carrots
SAFE	Safe
SLING	Sling
TNT1	TNT (1 stick)
TNT2	TNT (5 sticks)

► Analysis

This first solution uses subqueries. The inner `SELECT` statement does a simple retrieval to return the `vend_id` of the vendor that makes item `DTNTR`. That ID is the one used in the `WHERE` clause of the outer query so that all items produced by that vendor are retrieved. (You learned all about subqueries in Chapter 14, “Working with Subqueries.” Refer to that chapter for more information.)

Now look at the same query using a join:

► Input

```
SELECT p1.prod_id, p1.prod_name
FROM products AS p1, products AS p2
WHERE p1.vend_id = p2.vend_id
      AND p2.prod_id = 'DTNTR';
```

► Output

prod_id	prod_name
DTNTR	Detonator
FB	Bird seed
FC	Carrots
SAFE	Safe
SLING	Sling
TNT1	TNT (1 stick)
TNT2	TNT (5 sticks)

► Analysis

The two tables needed in this query are actually the same table, and so the `products` table appears in the `FROM` clause twice. Although this is perfectly legal, any references to the `products` table would be ambiguous because MySQL would not know to which instance of the `products` table you were referring.

To resolve this problem, you can use table aliases. The first occurrence of `products` has the alias `p1`, and the second has the alias `p2`. Now those aliases can be used as table

names. The `SELECT` statement, for example, uses the `p1` prefix to explicitly state the full name of the desired columns. If it doesn't, MySQL returns an error because there are two columns named `prod_id` and `prod_name`. It cannot know which one you want (even though, in truth, they are one and the same). The `WHERE` clause first joins the tables (by matching `vend_id` in `p1` to `vend_id` in `p2`), and then it filters the data by `prod_id` in the second table to return only the desired data.

Here is the same statement using `INNER JOIN` syntax:

► Input

```
SELECT p1.prod_id, p1.prod_name
FROM products AS p1
INNER JOIN products AS p2
  ON p1.vend_id = p2.vend_id
WHERE p2.prod_id = 'DTNTR';
```

Tip

Self-Joins Instead of Subqueries Self-joins are often used to replace statements using subqueries that retrieve data from the same table as the outer statement. Although the end result is the same, sometimes these joins execute far more quickly than do subqueries. It is usually worth experimenting with both to determine which performs better in a particular situation.

Natural Joins

Whenever tables are joined, at least one column appears in more than one table (the columns being joined). Standard joins (the inner joins you learned about in the previous chapter) return all data—even multiple occurrences of the same column. A *natural join* simply eliminates those multiple occurrences so only one of each column is returned.

How does it do this? The answer is it doesn't; you do it. A natural join is a join in which you select only columns that are unique. This is typically done using a wildcard (`SELECT *`) for one table and explicit subsets of the columns for all other tables. The following is an example:

► Input

```
SELECT c.*, o.order_num, o.order_date,
       oi.prod_id, oi.quantity, oi.item_price
FROM customers AS c, orders AS o, orderitems AS oi
WHERE c.cust_id = o.cust_id
      AND oi.order_num = o.order_num
      AND prod_id = 'FB';
```

► Analysis

In this example, a wildcard is used for the first table only. All other columns are explicitly listed so that no duplicate columns are retrieved.

Tip

Inner Joins and Natural Joins The truth is, every inner join you have created thus far is actually a natural join, and you will probably never even need an inner join that is not a natural join.

Outer Joins

Most joins relate rows in one table with rows in another. But occasionally, you want to include rows that have no related rows. For example, you might use joins to accomplish the following tasks:

- Count how many orders each customer placed, including customers who have yet to place an order
- List all products with order quantities, including products not ordered by anyone
- Calculate average sale sizes, taking into account customers who have not yet placed an order

In each of these examples, the join includes table rows that have no associated rows in the related table. This type of join is called an *outer join*.

The following `SELECT` statement is a simple inner join. It retrieves a list of all customers and their orders:

► Input

```
SELECT customers.cust_id, orders.order_num
FROM customers INNER JOIN orders
ON customers.cust_id = orders.cust_id;
```

Outer join syntax is similar. To retrieve a list of all customers, including those who have placed no orders, you can use the following:

► Input

```
SELECT customers.cust_id, orders.order_num
FROM customers LEFT OUTER JOIN orders
ON customers.cust_id = orders.cust_id;
```

► Output

cust_id	order_num
10001	20005
10001	20009
10002	NULL
10003	20006
10004	20007
10005	20008

 Analysis

Like the inner join shown in the previous chapter, this `SELECT` statement uses the keyword `OUTER JOIN` to specify the join type (instead of the `WHERE` clause). But unlike inner joins, which relate rows in both tables, outer joins also include rows with no related rows. When using `OUTER JOIN` syntax, you must use the `RIGHT` keyword or `LEFT` keyword to specify the table from which to include all rows; you use `RIGHT` for the one on the right of `OUTER JOIN` and `LEFT` for the one on the left. The previous example uses `LEFT OUTER JOIN` to select all the rows from the table on the left in the `FROM` clause (the `customers` table). To select all the rows from the table on the right, you use `RIGHT OUTER JOIN`, as shown in this example:

 Input

```
SELECT customers.cust_id, orders.order_num
FROM customers RIGHT OUTER JOIN orders
ON orders.cust_id = customers.cust_id;
```

Note

No `*=` For an `OUTER JOIN`, you need to use ANSI syntax. MySQL does not support the use of the simplified `*=` and `=*` syntax popularized by other DBMSs.

Tip

Outer Join Types There are two basic forms of outer joins: the left outer join and the right outer join. The only difference between them is the order of the tables that are being related. A left outer join can be turned into a right outer join simply by reversing the order of the tables in the `FROM` or `WHERE` clause. Therefore, the two types of outer join can be used interchangeably, and the decision about which one to use is based purely on convenience.

Using Joins with Aggregate Functions

As you learned in Chapter 12, “Summarizing Data,” aggregate functions are used to summarize data. Although all the examples of aggregate functions thus far have only summarized data from a single table, these functions can also be used with joins. To demonstrate this, let’s look at an example.

You want to retrieve a list of all customers and the number of orders that each has placed. The following code uses the `Count()` function to achieve this:

 Input

```
SELECT customers.cust_name,
       customers.cust_id,
       Count(orders.order_num) AS num_ord
FROM customers INNER JOIN orders
ON customers.cust_id = orders.cust_id
GROUP BY customers.cust_id;
```

► Output

cust_name	cust_id	num_ord
Coyote Inc.	10001	2
Wasicals	10003	1
Yosemite Place	10004	1
E Fudd	10005	1

► Analysis

This `SELECT` statement uses `INNER JOIN` to relate the `customers` and `orders` tables to each other. The `GROUP BY` clause groups the data by customer, and the function call `COUNT(orders.order_num)` counts the number of orders for each customer and returns it as `num_ord`.

Aggregate functions can be used just as easily with other join types. See the following example:

► Input

```
SELECT customers.cust_name,
       customers.cust_id,
       Count(orders.order_num) AS num_ord
FROM customers LEFT OUTER JOIN orders
ON customers.cust_id = orders.cust_id
GROUP BY customers.cust_id;
```

► Output

cust_name	cust_id	num_ord
Coyote Inc.	10001	2
Mouse House	10002	0
Wasicals	10003	1
Yosemite Place	10004	1
E Fudd	10005	1

► Analysis

This example uses a left outer join to include all customers—even those who have not placed any orders. The output shows that customer `Mouse House` (with 0 orders) is also included this time.

Using Joins and Join Conditions

Before wrapping up this two-chapter discussion of joins, it is worthwhile to summarize some key points regarding joins and their use:

- Pay careful attention to the type of join being used. More often than not, you'll want an inner join, but there are often valid uses for outer joins, too.

- Make sure you use the correct join condition, or you'll get incorrect data returned.
- Make sure you always provide a join condition, or you'll end up with the Cartesian product.
- You may include multiple tables in a join and even have different join types for each one. Although this is legal and often useful, make sure you test each join separately before testing them together. Doing so makes troubleshooting far simpler.

Summary

This chapter is a continuation of the previous chapter on joins. In this chapter you learned how and why to use aliases, and you learned about the different join types and various forms of syntax used with each of them. You also learned how to use aggregate functions with joins, and you learned some important do's and don'ts to keep in mind when working with joins.

Challenges

1. Write a SQL statement using `INNER JOIN` to retrieve customer names (`cust_name` in `Customers`) and all order numbers (`order_num` in `Orders`) for each.
2. Modify the SQL statement you just created to list all customers, even those with no orders.
3. Use `OUTER JOIN` to join the `Products` and `OrderItems` tables and return a sorted list of product names (`prod_name`) and the order numbers (`order_num`) associated with each.
4. Modify the SQL statement created in the previous challenge so that it returns the total number of orders for each item (as opposed to the order numbers).
5. Write a SQL statement that lists vendors (`vend_id` in `Vendors`) and the number of products they have available, including vendors with no products. You'll want to use `OUTER JOIN` and the `COUNT()` aggregate function to count the number of products for each of them in the `Products` table. Pay attention: The `vend_id` column appears in multiple tables, so any time you refer to it, you'll need to fully qualify it.